Cyrus Shobith Pinto - System Lead, IPC & Integration

STC89C52 Microcontroller Multi-Process Simulator - Student Study Guide

Role: Team Leader, Multi-Process Architect & Testing Specialist

1. What is TCP? (The Telephone Analogy for College Students)

TCP stands for **Transmission Control Protocol**.

Imagine you want to send a 500-page book to your friend across the city:

- **Without TCP (like UDP / mailing postcards):** You tear out each page and toss them into the wind. Some pages might arrive, some get lost in the rain, and page 50 arrives before page 1. Your friend gets a scrambled mess.
- **With TCP (a guaranteed telephone call):**

1. The 3-Way Handshake: Before sending any data, you pick up the phone:

- You: *"Hello, can you hear me?"* (SYN)
- CoreProcess: *"Yes, I hear you loud and clear. Can you hear me?"* (SYN-ACK)
- You: *"Great! Here comes the data."* (ACK)

2. Guaranteed Delivery & Perfect Order: Every single byte has a sequence number. If packet #3 drops, TCP automatically re-sends it. CoreProcess will NEVER receive packet #4 before packet #3.

3. What is a Port? Think of your computer as a large apartment building (IP: `127.0.0.1` / `localhost`). The port is the **apartment door number**:

- Apartment 5000 = `CoreProcess` (The CPU)
- Apartment 5001 = `LoggerProcess` (The Audit Log)
- Apartment 8080 = `UIProcess` (The Web Dashboard)

2. Is This the Simplest Connection Method for College PBL?

YES, 100%. Here is why this is the safest and cleanest approach:

- **Zero External Frameworks:** We did NOT use Spring Boot, React, Node.js, Vite, or npm. Complex web frameworks confuse beginners, cause dependency crashes, and make professors suspicious of "vibe coding".
- **Pure Standard Java (JDK):** Everything uses standard `java.net.Socket`, `java.io.BufferedReader`, and `com.sun.net.httpserver.HttpServer`. It runs out-of-the-box on every college lab PC.
- **The Entire Connection in Just 4 Lines of Code:**

```
// 1. Dial the CPU on Port 5000:
Socket socket = new Socket("localhost", 5000);

// 2. Speak into the phone (send "STEP"):
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
out.println("STEP");

// 3. Listen to the CPU's reply:
BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
String response = in.readLine(); // "STATE|PC=0002|A=10|..."

// 4. Hang up:
socket.close();
```

3. Files Owned & Managed by Cyrus

- `tests/TestCPU.java`: Automated unit test harness covering 14 discrete hardware test cases.
- `programs/QueueTest.java`: FIFO queue assembly program verifying data structure ordering.
- Multi-process integration code in `src/CoreProcess.java`: Implemented the multi-threaded client connection listener (`handleClient`) so both transient Web API requests and persistent Swing GUI sessions run without locking up the CPU.

4. Flowchart: Multi-Process IPC Topology

```
[ Browser / CLI (UIProcess) ]
             |
             | (TCP Port 5000: Packets like 'STEP', 'STATE', 'LOAD:74FE...')
             v
    +-------------------------------------------------------+
    |                 CoreProcess (Port 5000)               |
    |  - Multi-Threaded Socket Acceptor                     |
    |  - Fetch -> Decode -> Execute Loop                    |
    |  - CPU Registers & 256-Byte Memory                    |
    +-------------------------------------------------------+
             |
             | (TCP Port 5001: Streaming log lines '[STEP] PC=0002...')
             v
    +-------------------------------------------------------+
    |               LoggerProcess (Port 5001)               |
    |  - Asynchronous Audit Logger -> simulator.log         |
    +-------------------------------------------------------+
```

5. Step-by-Step Code Walkthrough (`TestCPU.java` & Integration)

- `TestCPU.java`: Instantiates isolated `Memory` and `CPU` objects. For each test case, it sets up initial register states, executes instruction bytes via `cpu.step()`, and evaluates boolean assertions using standard assertions without external dependencies like JUnit.
- **Handling Multi-Threading in** `CoreProcess.java`:

```
// In CoreProcess.java:
while (true) {
    Socket clientSocket = serverSocket.accept();
    new Thread(() -> handleClient(clientSocket)).start();
}
```

Every time a client connects (the web browser sending `cmd=STEP` or `SimulatorUI` connecting on startup), a lightweight worker thread is spawned to process the command and return the current CPU state immediately.

6. How Cyrus Can Add Features as College Teaches More

- **Adding a New Test Case:** Open `tests/TestCPU.java`. Add a new test method `testNewInstruction()`:

```
cpu.reset();
mem.write(0, (byte) 0x74); // MOV A, #0x50
mem.write(1, (byte) 0x50);
cpu.step();
assert regs.A == 0x50 : "Failed to load 0x50 into A";
```

- **Adding a New Socket Command:** Open `src/CoreProcess.java`. In `handleClient()`, add another `else if (verb.equals("NEW_CMD"))`.

7. Cyrus's 5 Hostile Examiner Questions & Defenses

Q1: "Why did you use TCP Sockets instead of Java Shared Memory or Static variables?"

Answer: *"Static variables and shared memory do not represent real-world embedded hardware. In the physical STC89C52, the microcontroller has its own dedicated silicon die and communicates with host computers via serial UART/SPI buses. TCP sockets enforce true process address space isolation; if the UI process crashes, the CPU continues running unaffected."*

Q2: "How do you prevent race conditions when two clients send commands at the exact same time?"

Answer: *"The CPU execution engine is synchronized around the core step cycle. Critical CPU state transitions (`cpu.step()` and memory writes) execute atomically so multiple incoming commands are serialized cleanly without corrupting registers."*

Q3: "Walk me through how `tests/TestCPU.java` works."

Answer: *"TestCPU runs 14 test cases covering every opcode (MOV, ADD, INC, ANL, SJMP, ENQUEUE, DEQUEUE, HALT), register boundary conditions (0xFF to 0x00 overflow wrap), flag arithmetic (CY, AC, OV, P), and FIFO ordering. It executes headlessly and prints a clean PASSED matrix with zero external dependencies."*

Q4: "What happens if a user inputs an invalid command over the socket?"

Answer: *"The server parses incoming lines using token splits. If the command verb is not in our recognized protocol set (STEP, RUN, STOP, RESET, LOAD, STATE, QUIT), CoreProcess responds with* `ERROR|UNKNOWN_COMMAND` *and retains its current state without throwing an unhandled exception."*

Q5: "What is your IPC packet protocol syntax?"

Answer: *"We use a pipe-delimited key-value string format. Commands are simple ASCII verbs like* `STEP` *or* `LOAD <hex>`*. Responses return* `STATE|PC=0000|A=00|CY=false|...` *followed by optional pipeline logs. This makes debugging transparent using simple telnet or netcat tools."*